2026-5-25 C++继承：下

- 4.派生类的默认成员函数
  - 4.1 4个常见的默认成员函数
  - 4.2 实现一个不能被继承的类
- 5. 继承与友元
- 6. 继承与静态成员
- 7.多继承及其菱形继承问题
  - 7.1 继承模型
  - 7.2 虚继承
  - 7.3 多继承中的指针偏移问题
  - 7.4 菱形虚拟继承示例--IO库
- 8. 继承和组合
  - 8.1 继承和组合

4.派生类的默认成员函数

4.1 4个常见的默认成员函数



在派生类中，我们类的六大成员函数是如何生成的呢?
我们以下面的函数为基类展开举例：

```
class Person {
public:
    Person(const char* name = "Peter")
        : _name(name)
    {
        cout << "Person()" << endl;
    }

    Person(const Person& p)
        : _name(p._name)
    {
        cout << "Person(const Person& p)" << endl;
    }

    Person& operator=(const Person& p) {
        cout << "Person& operator=(const Person& p)" << endl;
        if (this != &p)
            _name = p._name;
```

```
        return *this;
    }

    ~Person() {
        cout << "~Person()" << endl;
    }

protected:
    string _name;
};
```

1. 派生类的构造函数必须调用基类的构造函数初始化基类的那一部分。如果基类没有默认的构造函数，则必须在派生类构造函数的初始化列表阶段显示调用。

```
class Student : public Person {
public:
    // 无参构造函数
    Student()
        : Person()   // 调用基类的无参构造
        , _num(0)
    {
        cout << "Student()" << endl;
    }
    // 带参构造函数
    Student(const char* name, int num)
        : Person(name)
        , _num(num)
    {
        cout << "Student(const char* name, int num)" << endl;
    }
 protected:
    int _num;
};
```

2. 派生类的拷贝构造必须调用基类的拷贝构造完成基类的拷贝初始化。

```
class Student : public Person {
public:

    // 拷贝构造
    Student(const Student& s)
        : Person(s)
        , _num(s._num)
```

```
    {
        cout << "Student(const Student& s)" << endl;
    }
    protected:
    int _num;
};
```

3. 派生类的operator=必须调用基类的operator=完成基类的复制。需要注意的是派生类的operator=隐藏了基类的operator=，所以显示调用基类的operator=，需要指定基类的作用域。

```
class Student : public Person {
public:

    // 赋值运算符重载
    Student& operator=(const Student& s) {
        cout << "Student& operator=(const Student& s)" << endl;
        if (this != &s) {
            //构成隐藏所以需要显示调用
            Person::operator=(s);
            _num = s._num;
        }
        return *this;
    }
    protected:
    int _num;
};
```

4. 派生类的析构函数会在被调用完成后自动调用基类的析构函数清理基类成员。因为这样才能保证派生类对象先清理派生类成员。因为这样才能保证派生类对象先清理派生类对象在清理基类对象的顺序。为什么一定要是这个顺序呢？因为如果我们假想一下如果我们先删除了基类万一你再调用一下派生类那不就出问题了吗？我们先删除派生类再调用基类是不会有问题的。

```
class Student : public Person {
public:
    ~Student() {
        cout << "~Student()" << endl;
    }

protected:
```

```
    int _num;
};
```

5. 派生类对象初始化先调用基类构造再调用派生类构造
6. 派生类对象析构清理先调用派生类析构再调用基类析构
7. 因为多态中一些场景析构函数需要构成重写，重写的条件之一是函数名相同。那么编译器会对析构函数名进行特殊处理，处理成destructor(),所以基类的析构函数不加virtual的情况下，派生类析构函数和基类析构函数构成隐藏关系。

```
int main() {
    Student s1;              // 调用无参构造
    Student s2("张三", 18); // 调用带参构造
    Student s3(s2);          // 调用拷贝构造
    s1 = s3;                 // 调用赋值运算符重载
    return 0;
}
```

4.2 实现一个不能被继承的类

方法1：基类的构造函数私有，派生类的构成必须调用基类的构造函数，但是基类的构成函数私有化以后，派生类看不见就不能调用了，那么派生类就无法实例化出对象。

```
//C++98的方法
class Base
{
public:
    void func1() { cout << "haha" << endl; }
protected:
    int a = 1;
private:
    Base() {};
};
class Derive : public Base {
    void func2() { cout << "xixi" << endl; }
protected:
    int b = 2;
};
int main() {
    Base b;//在定义函数的时候出错
    Derive d;
```

```
    return 0;
}
```

方法2：C++新增了一个final关键字，final修改基类，派生类就不不能继承了。

```
//C++11的方法
class Base final
{
public:
    void func1() { cout << "haha" << endl; }
protected:
    int a = 1;
private:
    Base() {};
};
class Derive : public Base {//在继承的时候出错，不允许继承
    void func2() { cout << "xixi" << endl; }
protected:
    int b = 2;
};
int main() {
    Derive d;
    return 0;
}
```

5. 继承与友元

友元关系不能继承，也就是说基类友元不能访问派生类私有成员和保护成员。（你爸爸的朋友不是你的朋友）

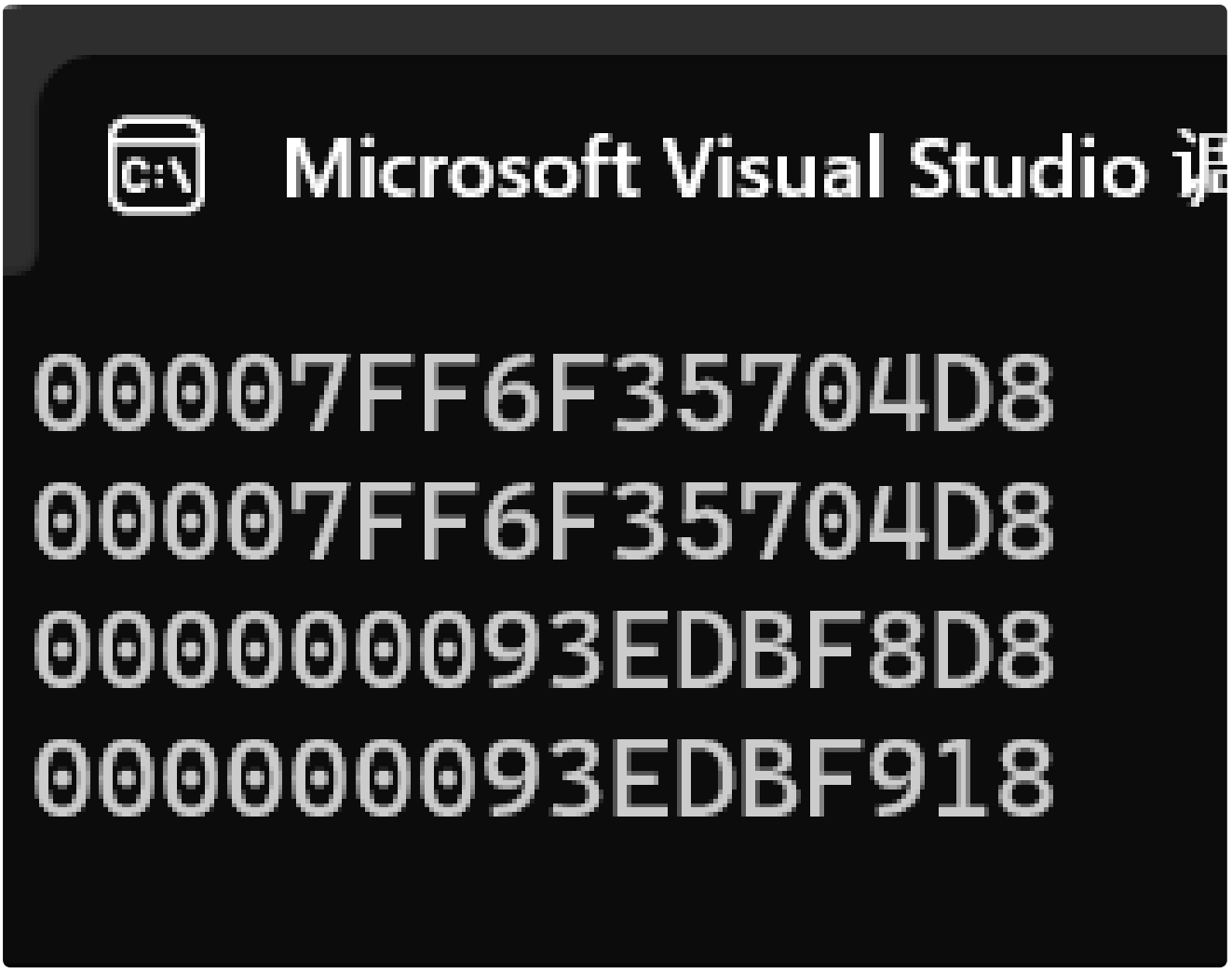
```
class Student;//因为要在Person里面使用Srudent
//Student定义在Person的后面如果不声明一下编译器是不认识的
class Person {
public:
    friend void Display(const Person& p, const Student& s);
protected:
    string name;
};
class Student :public Person {
protected:
    int num;
};
void Display(const Person& p, const Student& s) {
```

```
    cout << p.name << endl;
    cout << s.num << endl;//基类的友元不能访问派生类的私有成员变量
    //这里会报错说明s.num不可访问
}
int main() {
    Student s;
    Person p;
    Display(p,s);
    return 0;
}
```

6. 继承与静态成员

基类定义了static静态成员，则整个继承体系里面只有一个这样的成员。无论派生出多少个派生类，都只有一个static成员实例。

```
class Person {
public:
    string name;
    static int count;//这里只是声明不是定义
protected:
    static int _count;
};
int Person::count = 0;
int Person::_count = 0;
class Student :public Person {
protected:
    int num;
};
int main() {
    Person p;
    Student s;
    //可以看到，将成员变量置为静态的访问的都是同一个地址
    cout << &p.count << endl;
    cout << &s.count << endl;
    //如果设为保护就不可以访问
    //cout << &p._count << endl;
    //cout << &s._count << endl;
    //如果该成员不是静态的,那么它们的地址是不一样的
    cout << &p.name << endl;
    cout << &s.name << endl;
    return 0;
}
```



为什么一定要有下面这段代码程序才能正常运行?

```
int Person::count = 0;
```

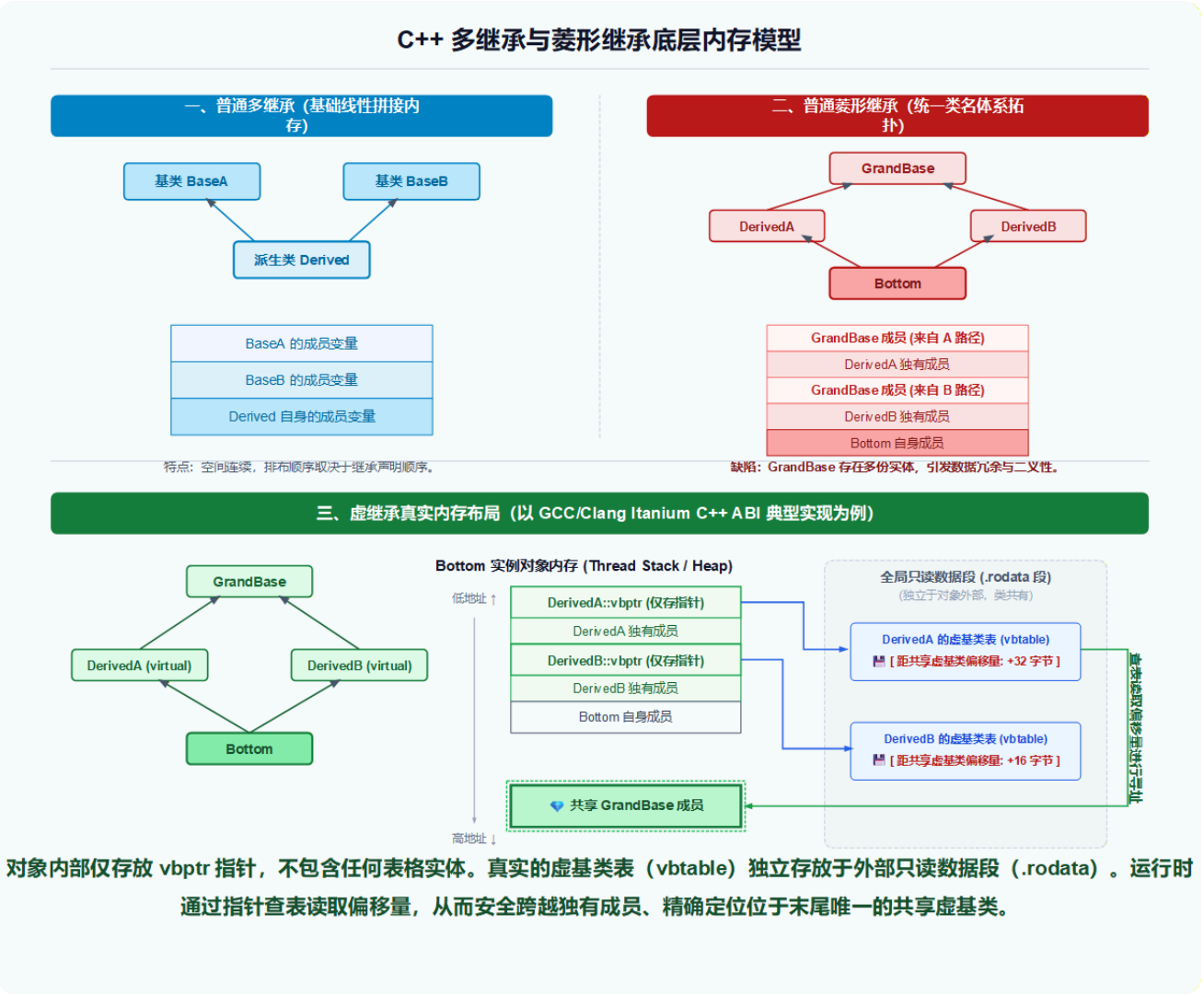
因为静态成员变量属于类本身，并不属于某个对象必须在类外单独定义一次才会分配内存并存在。
那为什么不能在类内直接定义呢？原因在于：

1. **声明 ≠ 定义**：类内的 `static int count;` 只是声明，表示“有这个变量”，但不分配内存。
2. **避免重复定义**：头文件可能被多个 `.cpp` 文件包含，如果在类内定义，会导致链接时出现多个重复定义。
3. **独立存储空间**：静态成员属于类本身，不属于任何对象，必须在类外唯一地定义一次，以在静态存储区分配内存。

7.多继承及其菱形继承问题

7.1 继承模型

单继承：一个派生类只有一个直接基类时称这个继承关系为单继承
多继承：一个派生类有两个或者两个以上直接基类时称这个继承关系为多继承，多继承对象在内存中的模型是，先继承的基类在前面，后继承的基类在后面，派生类成员放到最后面。
菱形继承：菱形继承是多继承的一种特殊情况，从下面的对象成员模型构造，可以看出菱形继承有**数据冗余和二义性**的问题。



示例代码如下：

```
class Person {
public:
    string _name;//姓名
};
class Student :public Person
{
protected:
    int _num;
};
class Teacher :public Person
{
protected:
    int _id;//职工编号
};
```

```
class Assistant :public Student, public Teacher//这里会造成数据冗余，Person这个类继承了俩遍
{
protected:
    string _Course;//课程
};
int main() {
    Assistant a;
    //a._name = "haha";//这个时候就体现出二义性了，这个name不知道是给老师还是给学生
    //我们想解决也行，指定类域嘛,但是很麻烦对不对，没关系我们还有办法：虚继承
    a.Student::_name = "张三";
    a.Person::_name = "张老师";
    return 0;
}
```

支持多继承就一定会有菱形继承，Java就直接不支持多继承，规避掉了这里的问题，所以实践中不建议设计出菱形继承这样的模型。

7.2 虚继承

有了菱形继承就有菱形虚拟继承，底层实现就很复杂，性能也会有损失，一般情况下建议不要设计出菱形继承。

```
class Person {
public:
    string _name;//姓名
};
class Student :virtual public Person
{
protected:
    int _num;
};
class Teacher :virtual public Person
{
protected:
    int _id;//职工编号
};
class Assistant :public Student, public Teacher//这里会造成数据冗余，Person这个类继承了俩遍
{
protected:
    string _Course;//课程
};
```

```
int main() {
    Assistant a;
    //使用虚拟继承可以解决数据冗余和二义性
    a._name = "haha";
    return 0;
}
```

来一个小测试：

```
class Person {
public:
    Person() : _name("默认") {}
    Person(const char* name)
        :_name(name) {

    }
    string _name;//姓名
};
class Student :virtual public Person
{
public:
    Student(const char* name, int num)
        :Person(name)
        , _num(num) {

    }
protected:
    int _num;
};
class Teacher :virtual public Person
{
public:
    Teacher(const char* name, int id)
        :Person(name)
        , _id(id) {

    }
protected:
    int _id;
};
//不使用菱形继承
class Assistant :public Student, public Teacher//这里会造成数据冗余，Person这个类继承了俩遍
{
public:
```

```
    Assistant(const char* name1, const char* name2, const char* name3)
        :Person(name1)
        , Student(name2,888)
        , Teacher(name3,666) {
    }
protected:
    string _Course;//课程
};
int main() {
    Assistant a("张三", "张同学", "张老师");
    cout << a._name << endl;//这里调用的_name是Person的名字"张三"
    cout << a.Student::_name << endl;//这样也是"张三"
    cout << a.Teacher::_name << endl;//这样也是"张三"
    return 0;
}
```

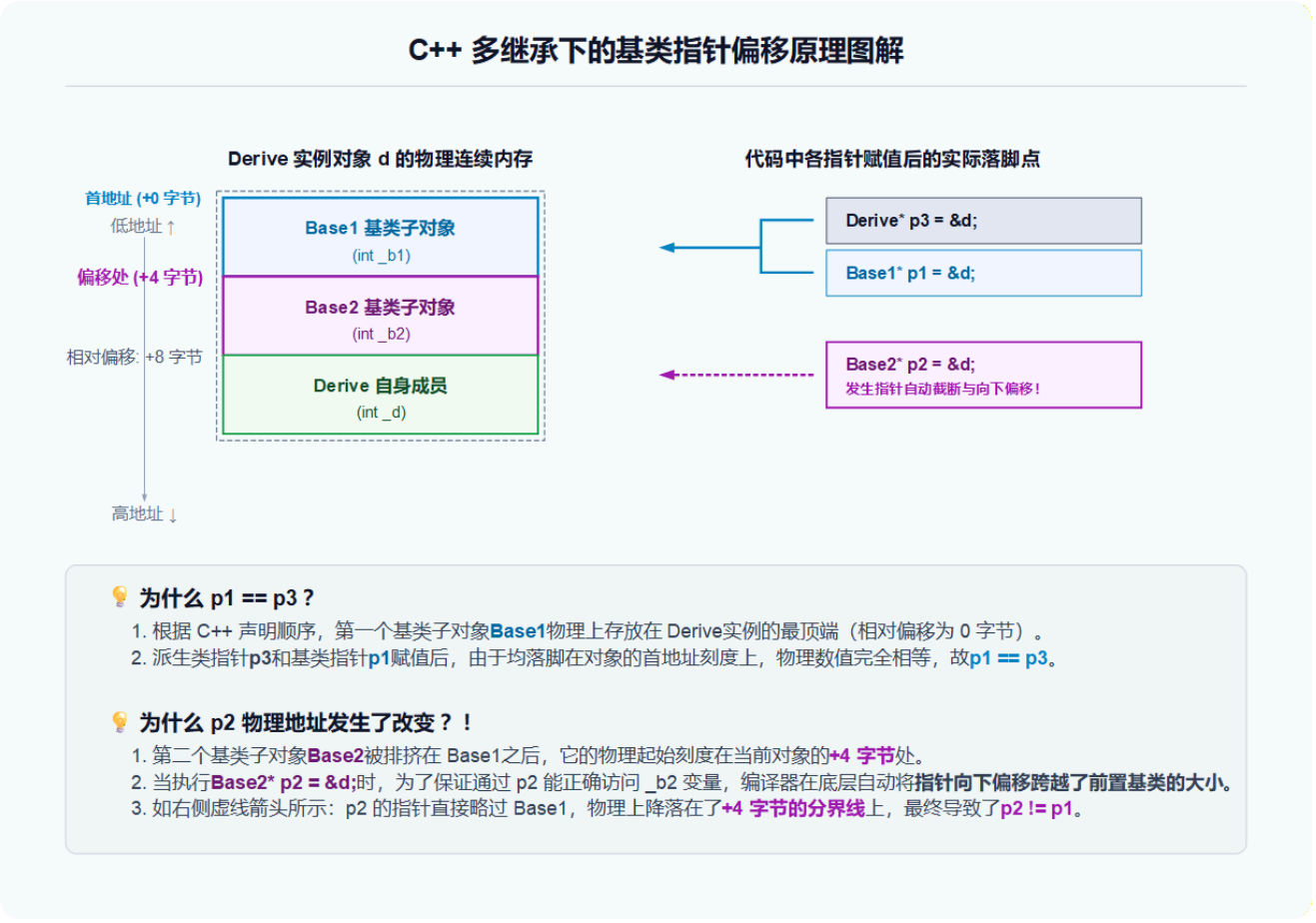
哎，张老师和张同学无了，调不出来了是为什么呢?
虚继承强制 `Person` 在 `Assistant` 中只存在一份，因此 `Student` 和 `Teacher` 的构造函数中传给 `Person` 的不同名字被合并为同一个，无法区分。 要保留“张同学/张老师”，必须去掉虚继承（就是去掉virtual但是数据冗余的问题会保留)，或把角色名单独存在 `Student` / `Teacher` 自己的成员中。 （这个稍微解释一下就是不让张老师和张同学这两个名字从Person中继承，而是单独定义)。

7.3 多继承中的指针偏移问题

多继承这种指针偏移下列说法正确的是( )
A.P1 `==` p2 `==` p3 B.P1 `<` p2 `<` p3 C.P1 `==` p3 `!=` p2 D.P1 `!=` p2 `!=` p3

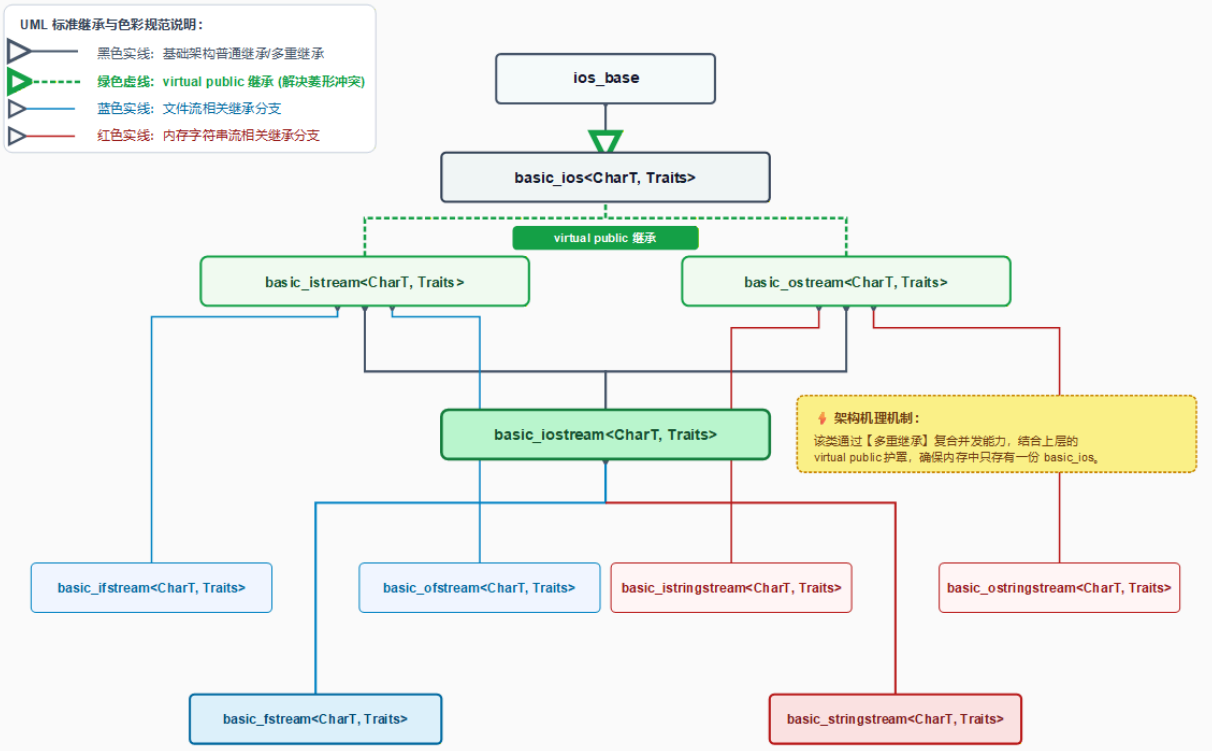
```
class Base1 { public: int _b1; };
class Base2 { public: int _b2; };
class Derive :public Base1, public Base2 { public:int _d; };

int main() {
    Derive d;
    Base1* p1 = &d;
    Base2* p2 = &d;
    Derive* p3 = &d;
    cout << p1 << endl;
    cout << p2 << endl;
    cout << p3 << endl;
    return 0;
}
```



7.4 菱形虚拟继承示例--IO库

C++ 标准 I/O 库流类继承体系谱系图



标准常用特化别名（char 特化）

```cpp
typedef basic_ios<char> ios;
typedef basic_istream<char> istream;
typedef basic_ostream<char> ostream;
typedef basic_iostream<char> iostream;

// 磁盘物理文件流别名
typedef basic_ifstream<char> ifstream;
typedef basic_ofstream<char> ofstream;
typedef basic_fstream<char> fstream;

// 内存字符串流别名
typedef basic_istringstream<char> istringstream;
typedef basic_ostringstream<char> ostringstream;
typedef basic_stringstream<char> stringstream;
```

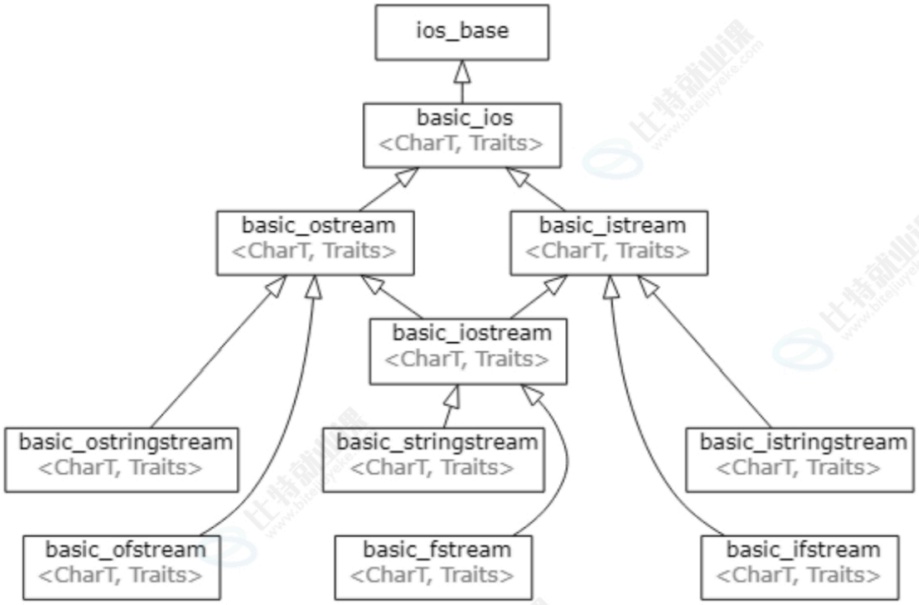
```cpp
/* C++ 原生标准库内核级源码定义（规范特化形参声明） */
// 已根据 ISO C++ 标准规范修正类名拼写与继承关系，确保与标准库定义完全一致

// 1. 输入流模板虚派生实现
template<class CharT, class Traits = std::char_traits<CharT>>
class basic_istream : virtual public basic_ios<CharT, Traits> { ... };

// 2. 输出流模板虚派生实现
template<class CharT, class Traits = std::char_traits<CharT>>
class basic_ostream : virtual public basic_ios<CharT, Traits> { ... };

// 3. 多重继承交汇点定义
template<class CharT, class Traits = std::char_traits<CharT>>
class basic_iostream : public basic_istream<CharT, Traits>,
                       public basic_ostream<CharT, Traits> { ... };
```

因为 basic_istream 和 basic_ostream 锁定了 virtual 关键字，最终通过多重继承派生的落地实体（如 fstream）内部只会持有一份无冲突的 basic_ios 数据结构。




```
template<class CharT, class Traits = std::char_traits<CharT>>
class basic_ostream : virtual public std::basic_ios<CharT, Traits>
{};

template<class CharT, class Traits = std::char_traits<CharT>>
class basic_istream : virtual public std::basic_ios<CharT, Traits>
{};
```

8. 继承和组合

8.1 继承和组合

1. public继承是一种is-a的关系。也就是说每个派生类对象都是一个基类对象。
2. 组合是一种has-a的关系。假设B组合了A，每个B对象都有一个A对象。
3. 继承允许我们根据基类来定义派生类的实现。这种通过派生类的复用通常称为白箱复用（是相对与可视化而言的），在继承方式中，基类的内部细节对派生类可见。继承一定程度上破化了基类的封装，基类的改变对派生类的影响很大。派生类和基类间的依赖关系很强，耦合度高。
4. 对象组合是类继承之外的另一种复用选择。新的更复杂的功能可以通过组装或者组合对象来获得。对象组合要求被组合的对象具有良好的接口定义。这种复用风格被称为黑箱复用，因为对象的内部细节是不可见的。对象只能以黑箱的形式出现，组合类之间没有很强的依赖关系，耦合度低。**从一定程度上可以保证你的每个类都被封装**。
5. 我们在日常使用中，优先使用组合而不是继承。组合的耦合度低，代码维护性好。当然也要分情况来看，但是当组合和继承都可以使用的时候优先使用组合。

```
// ========== 被复用的组件 ==========
class Engine {
public:
    void start() const {
        std::cout << "Engine started." << std::endl;
    }
};

class Wheel {
public:
    void roll() const {
        std::cout << "Wheel rolls." << std::endl;
    }
};

// ========== 1. 继承方式：Car IS-A Engine（错误语义，仅为示例） ==========
// 注意：这种继承关系在语义上是不合理的（车不是发动机）
class CarInherit : public Engine, public Wheel {
public:
    void drive() {
        start();   // 来自 Engine
        roll();    // 来自 Wheel
        std::cout << "Car is driving (by inheritance)." << std::endl;
    }
};

// ========== 2. 组合方式：Car HAS-A Engine（正确语义） ==========
class CarCompose {
private:
    Engine engine_;   // 组合：Car 拥有 Engine
    Wheel  wheel_;    // 组合：Car 拥有 Wheel
public:
    void drive() {
        engine_.start();
        wheel_.roll();
        std::cout << "Car is driving (by composition)." << std::endl;
    }
};

// ========== 3. 组合 + 初始化：更灵活的依赖注入（推荐） ==========
class CarFlex {
private:
    Engine& engine_;   // 引用：依赖外部传入
    Wheel&  wheel_;    // 引用：依赖外部传入
public:
```

```
    // 构造函数注入依赖
    CarFlex(Engine& e, Wheel& w)
        : engine_(e), wheel_(w) {}

    void drive() {
        engine_.start();
        wheel_.roll();
        std::cout << "Car is driving (flexible composition)." << std::endl;
    }
};

// ========== 测试 ==========
int main() {
    CarInherit ci;
    ci.drive();

    CarCompose cc;
    cc.drive();

    Engine e;
    Wheel w;
    CarFlex cf(e, w);
    cf.drive();
    return 0;
}
```